SIMATS ENGINEERING
ASSESSMENT TOOL 1: Experiments

COURSE NAME: COMPUTER VISION COURSE CODE:ITA0510 Slot A

COURSE OUTCOME COVERED

CO3: *Analyze and extract image features using detection and matching techniques for effective image representation and comparison.* (BTL 4)

Assessment Tool Weightage: 40%

QUESTIONS:3 Total Marks 30 Duration :60 Minutes

Annexure A

Experiments

Code of Conduct:

I S.Varsha Reddy(192572177) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:Varsha Reddy

Rubrics for Evaluation

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)	*Marks(100)*
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts	
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation	
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools	
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results	
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation	

1. Preprocessing Impact Analysis

Design and perform an experiment to study the impact of preprocessing techniques such as noise removal and normalization on image quality and feature detection. Explain the theoretical concepts behind preprocessing methods and justify the selection of techniques used. Implement the preprocessing process using suitable software/tools, document the workflow and outputs systematically, and analyze how preprocessing influences feature detection accuracy and reliability.

Ans:Preprocessing Impact Analysis

Aim

To study the impact of image preprocessing techniques such as noise removal and normalization on image quality and feature detection accuracy.

Theory

Image preprocessing improves image quality before feature extraction by removing noise and enhancing contrast.

Noise Removal

Noise degrades image quality and affects feature detection.

Median Filter

- Removes Salt-and-Pepper noise.
- Preserves image edges.
- Produces a smoother image.

Normalization

Normalization scales pixel values to a standard range (0–255), improving image consistency and contrast.

Feature Detection

Canny Edge Detection is used to detect object boundaries by identifying significant intensity changes in the image.

Software Used

- Python 3.x
- OpenCV
- NumPy
- Matplotlib

Algorithm

1. Read the input image.
2. Convert the image to grayscale.
3. Add Salt-and-Pepper noise.
4. Apply the Median Filter.
5. Normalize the filtered image.
6. Detect edges using the Canny Edge Detector.
7. Compare the results before and after preprocessing.

Python Code

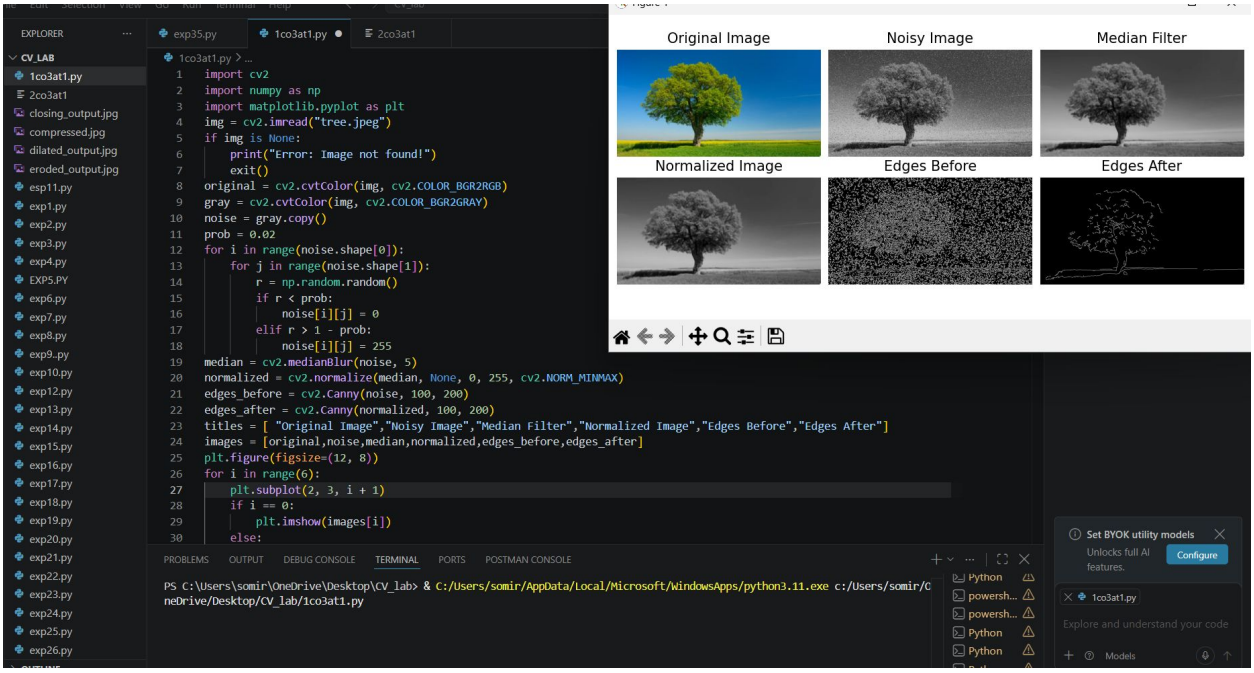
```
import cv2
import numpy as np
import matplotlib.pyplot as plt
img = cv2.imread("tree.jpeg")
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
noise = gray.copy()
prob = 0.02
for i in range(noise.shape[0]):
    for j in range(noise.shape[1]):
```

```
            r = np.random.random()
            if r < prob:
                noise[i][j] = 0
            elif r > 1 - prob:
                noise[i][j] = 255
median = cv2.medianBlur(noise, 5)
normalized = cv2.normalize(median, None, 0, 255, cv2.NORM_MINMAX)
edges_before = cv2.Canny(noise, 100, 200)
edges_after = cv2.Canny(normalized, 100, 200)
titles = ['Original','Noisy','Median Filter', 'Normalized','Edges Before','Edges After']
images = [gray, noise, median, normalized, edges_before, edges_after]
plt.figure(figsize=(12,8))
for i in range(6):
    plt.subplot(2,3,i+1)
    plt.imshow(images[i], cmap='gray')
    plt.title(titles[i])
    plt.axis('off')
plt.show()
```

Output

The program displays:

- Original Image
- Noisy Image
- Median Filtered Image
- Normalized Image
- Edge Detection Before Preprocessing
- Edge Detection After Preprocessing



Result

The experiment successfully demonstrated that Median Filtering removes noise while preserving edges, and Normalization improves image contrast. As a result, Canny Edge Detection produced clearer and more accurate edges after preprocessing, proving that preprocessing significantly improves image quality and feature detection accuracy.

2. Image Representation Study

Conduct an experiment to convert a given image into grayscale and binary representations. Explain the significance of different image representations in computer vision applications. Design the conversion procedure using appropriate tools, document the generated outputs clearly, and analyze how image representation affects edge detection and feature extraction performance.

Ans:

Aim

To convert a given image into grayscale and binary representations and analyze their impact on edge detection and feature extraction using OpenCV.

Theory

Image representation is the process of expressing an image in different formats for analysis and processing. Different representations simplify image processing tasks and improve computational efficiency.

Grayscale Image

- Contains intensity values ranging from 0 to 255.
- Reduces image complexity by removing color information.
- Commonly used in image processing and computer vision.

Binary Image

- Contains only two pixel values (0 and 255).
- Generated using thresholding.
- Useful for object segmentation, shape analysis, and pattern recognition.

Edge Detection

Edge detection identifies object boundaries by detecting sudden changes in pixel intensity. Different image representations influence the quality of detected edges and extracted features.

Software Used

- Python 3.x
- OpenCV
- NumPy
- Matplotlib

Algorithm

1. Read the input image.
2. Convert the image into grayscale.
3. Apply binary thresholding to create a binary image.
4. Perform Canny edge detection on the grayscale image.
5. Perform Canny edge detection on the binary image.
6. Display all output images.
7. Compare the edge detection results.
8. Stop.

Python Code

```
import cv2

import matplotlib.pyplot as plt

img = cv2.imread("tree.jpeg")

gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)_, binary = cv2.threshold(gray, 127, 255, cv2.THRESH_BINARY)

edges_gray = cv2.Canny(gray, 100, 200)

edges_binary = cv2.Canny(binary, 100, 200)

titles = ['Original Image', 'Grayscale Image','Binary Image', 'Edges (Grayscale)',

          'Edges (Binary)']

images = [cv2.cvtColor(img, cv2.COLOR_BGR2RGB), gray, binary,  edges_gray, edges_binary]
```

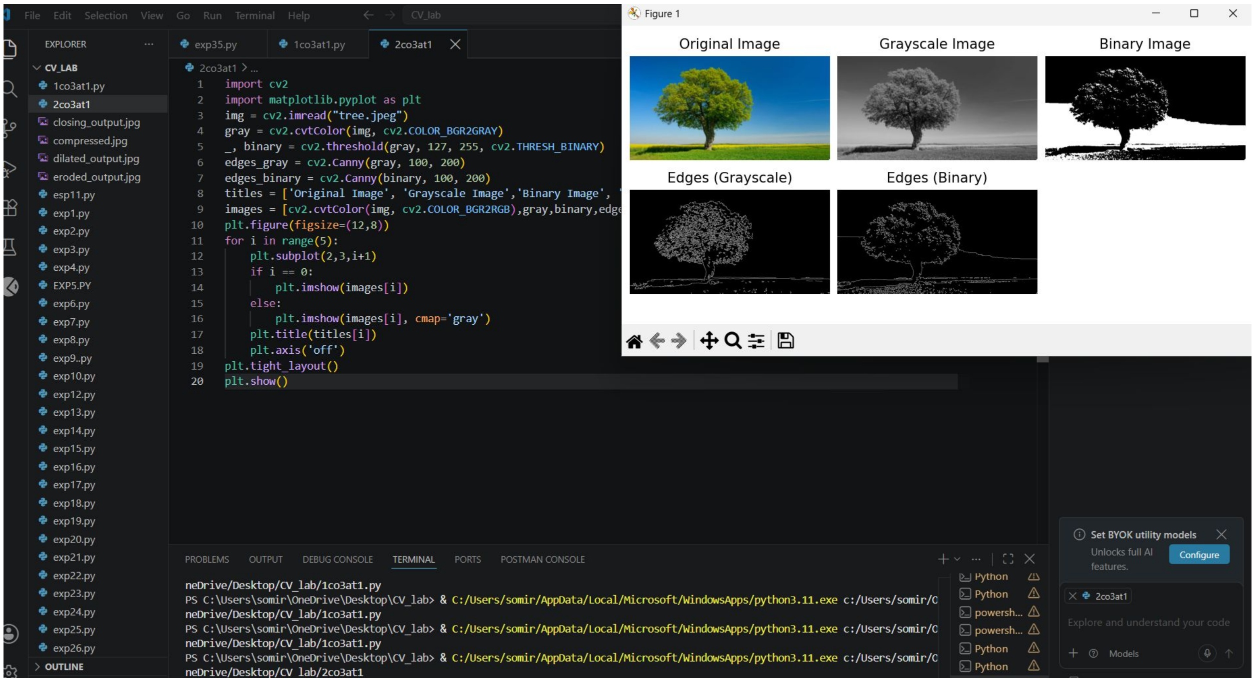
```
plt.figure(figsize=(12,8))
for i in range(5):
    plt.subplot(2,3,i+1)
    if i == 0:
        plt.imshow(images[i])
    else:
        plt.imshow(images[i], cmap='gray')
    plt.title(titles[i])
    plt.axis('off')
plt.tight_layout()
plt.show()
```

Output

Paste the screenshot of the program output here.

The output displays:

- Original Image
- Grayscale Image
- Binary Image
- Edge Detection on Grayscale Image
- Edge Detection on Binary Image



Result

The experiment successfully converted the input image into grayscale and binary representations. The grayscale image preserved more intensity information, resulting in better edge detection and feature extraction. The binary image simplified the image by separating the foreground from the background, making it suitable for segmentation and object recognition tasks. Thus, different image representations significantly affect the performance of edge detection and feature extraction in computer vision applications.

3. Edge Detection Comparison

Design an experiment to compare Sobel and Canny edge detection techniques on the same input image. Explain the underlying concepts and operational principles of both methods. Apply the techniques using suitable tools/software, document the detected edges systematically, and analyze the comparative effectiveness of each method for different image conditions and applications.

Ans:

Edge Detection Comparison

Aim

To compare Sobel and Canny edge detection techniques on the same input image and analyze their effectiveness under different image conditions.

Theory

Sobel Edge Detection calculates the intensity gradient in horizontal and vertical directions to identify edges. It is simple and useful for detecting basic edges.

Canny Edge Detection is a multi-stage method that includes noise reduction, gradient calculation, non-maximum suppression, double thresholding, and edge tracking. It generally produces thinner and cleaner edges.

Comparison

Feature	Sobel	Canny
Method	Gradient-based	Multi-stage
Noise Handling	Moderate	Better
Edge Thickness	Thicker	Thin
Accuracy	Moderate	High
Processing	Faster	More computational

Software Used

- Python 3.x
- OpenCV
- NumPy
- Matplotlib

Algorithm

1. Read the input image.
2. Convert the image to grayscale.
3. Apply Sobel filtering in the X and Y directions.
4. Combine the Sobel gradients to obtain the edge image.
5. Apply the Canny Edge Detection algorithm.
6. Display the original, Sobel, and Canny results.
7. Compare the detected edges.
8. Stop.

Python Code

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
img = cv2.imread("treee.jpeg")
if img is None:
    print("Error: Image not found!")
    exit()
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
sobel_x = cv2.Sobel(gray, cv2.CV_64F, 1, 0, ksize=3)
sobel_y = cv2.Sobel(gray, cv2.CV_64F, 0, 1, ksize=3)
```

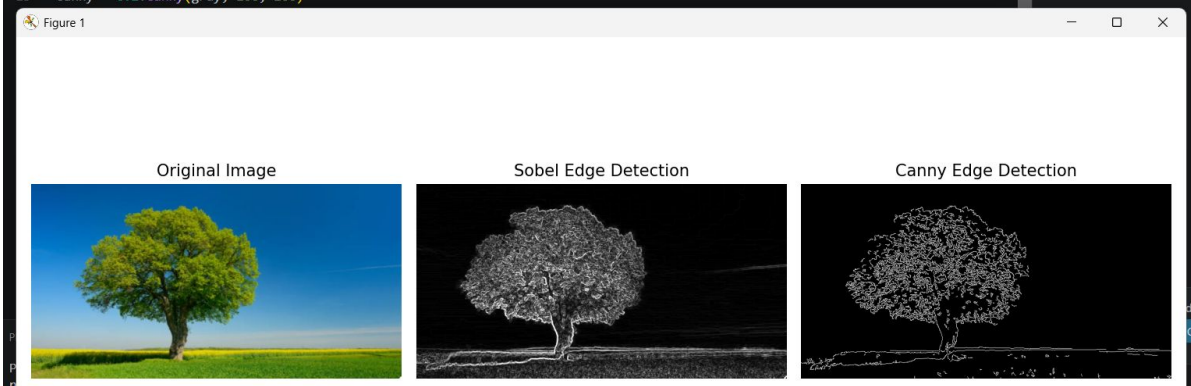
```
sobel = cv2.magnitude( np.float32(sobel_x), np.float32(sobel_y))
sobel = cv2.convertScaleAbs(sobel)
canny = cv2.Canny(gray, 100, 200)
original = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
titles = [  "Original Image","Sobel Edge Detection", "Canny Edge Detection"]
images = [ original,sobel,canny]
plt.figure(figsize=(12, 5))
for i in range(3):
    plt.subplot(1, 3, i + 1)
    if i == 0:
        plt.imshow(images[i])
    else:
        plt.imshow(images[i], cmap="gray")
    plt.title(titles[i])
    plt.axis("off")
plt.tight_layout()
plt.show()
```

Output

The output displays:

1. **Original Color Image**
2. **Sobel Edge Detection**
3. **Canny Edge Detection**

```python
import cv2
import numpy as np
import matplotlib.pyplot as plt
img = cv2.imread("tree.jpeg")
if img is None:
    print("Error: Image not found!")
    exit()
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
sobel_x = cv2.Sobel(gray, cv2.CV_64F, 1, 0, ksize=3)
sobel_y = cv2.Sobel(gray, cv2.CV_64F, 0, 1, ksize=3)
sobel = cv2.magnitude(np.float32(sobel_x),np.float32(sobel_y))
sobel = cv2.convertScaleAbs(sobel)
canny = cv2.Canny(gray, 100, 200)
```



Result

The experiment successfully compared Sobel and Canny edge detection techniques. Sobel detects edges quickly but may produce thicker and noisy edges. Canny provides clearer, thinner, and more accurate edges due to its multi-stage process. Therefore, Sobel is suitable for simple and fast edge detection, while Canny is more effective for applications requiring accurate object boundaries, such as object detection and image segmentation.